A PROJECT REPORT ON

" Room Finder: Rental Accommodation System "

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS

DIPLOMA IN COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SUBMITTED TO

GOVERNMENT POLYTECHNIC GULZARBAGH, PATNA– 07

SUBMITTED BY

NAME	ENROLLMENT NO.
Sandip Sah	1181823024
Saurabh Kumar	1181823048
Shivam Kumar	1181823601
Ankesh Bharti	1181823602

GUIDED BY:

Academic Year:-

2023– 2026

DECLARATION

We,

Sandip Sah (Roll no: 511181823024), Saurabh Kumar (Roll no: 511181823048), Shivam Kumar (Roll no: 511181823601), Ankesh Bharti (Roll no: 511181823602),

hereby declare that the Project Report entitled **"Room Finder: Rental Accommodation System"** is the result of our own work and has been carried out under the guidance of Faculties of Department of Computer Science and Engineering, Government Polytechnic Patna 7.

DATE:

PLACE: GOVERNMENT POLYTECHNIC GULZARBAGH, PATNA– 07

SIGNATURE OF THE CANDIDATES

Head of the Department

Submitted for Viva Voce Examination held on :

Internal Examiner **External Examiner**



GOVERNMENT POLYTECHNIC GULZARBAGH, PATNA-07

(STATE BOARD OF TECHNICAL EDUCATION, GOVERNMENT OF BIHAR)

(DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING)

CERTIFICATE

This is to certify that the Project entitled **"Room Finder: Rental Accommodation System"** has been successfully completed and submitted by the following students of **Diploma in Computer Science & Engineering**, Government Polytechnic Patna-7, in partial fulfilment of the requirements for the award of **Diploma in Computer Science & Engineering** under the **State Board of Technical Education (SBTE), Bihar**, during the academic session **2023-2026**.

S.No.	Student Name	Roll No.	Semester
01	Sandip Sah	511181823024	5thSem
02	Saurabh Kumar	511181823048	5thSem
03	Shivam Kumar	511181823601	5thSem
04	Ankesh Bharti	511181823602	5thSem

(Project Guide) **(Head of Department)**

ACKNOWLEDGMENT

We would like to express our sincere gratitude to all those who supported and guided us throughout the completion of this project. First and foremost, we are deeply thankful to our project guide for their valuable guidance, continuous encouragement, and helpful suggestions. Their constant support and technical advice played a crucial role in the successful completion of this project.

We are also grateful to the faculty members of the Department of Computer Science & Engineering, Government Polytechnic Patna 7, for providing us with the knowledge and laboratory resources required to complete this project. Their lectures and practical sessions helped us build a strong foundation in modern web technologies, database management, and system architecture.

We would like to thank our friends and classmates for their valuable cooperation and support. Their constructive discussions, peer reviews, and suggestions helped us improve the features of our application and overcome various design challenges during development. Working together and sharing technical ideas made the learning experience both productive and enjoyable.

We are especially thankful to our families for their constant motivation, encouragement, and moral support throughout our diploma journey. Their belief in our potential gave us the confidence to pursue this project with dedication.

Finally, we extend our heartfelt thanks to everyone who directly or indirectly contributed to the completion of this project. Developing the Room Finder platform has been a tremendous learning experience, and we are grateful for all the support we received.

ABSTRACT

The Room Finder System is a full-stack web application engineered to solve the persistent challenges associated with discovering and listing rental accommodation across Bihar. Every year, thousands of students and working professionals migrate to educational and administrative hubs such as Patna, Gaya, Darbhanga, Bhagalpur, and Muzaffarpur. Finding affordable rooms, paying-guest (PG) units, hostels, or flats traditionally involves unreliable physical searches, misleading hoardings, or middleman exploitation involving exorbitant brokerage fees.

Room Finder provides a centralized digital platform connecting prospective tenants directly with genuine property owners. Public visitors can freely search and filter accommodations by district, locality, room category, and budget range without mandatory account registration. Detailed listings present key property specifications, verified rental charges, deposit amounts, available amenities (such as WiFi, RO drinking water, power backup, and attached washrooms), and high-resolution photographs. A dedicated one-click contact feature enables direct communication with owners.

For property owners, the system delivers an intuitive listing management portal allowing them to post, update, and manage accommodations with multi-image uploads. Administrative controls ensure content integrity through moderation tools capable of removing misleading listings and regulating user accounts.

The system is constructed using HTML5, CSS3, and JavaScript on the client side, Node.js with Express.js on the backend, and MongoDB Atlas for flexible cloud document storage. Security is enforced through bcrypt password hashing, JSON Web Tokens stored in httpOnly cookies, and strict input sanitization. Overall, Room Finder presents a transparent, secure, and user-friendly solution to the student accommodation crisis in Bihar.

TABLE OF CONTENTS

CONTENT	PAGE NO.
Cover Page	01
Declaration	02
Certificate	03
Acknowledgement	04
Abstract	05
Table of Contents	06
Chapter 1 – Introduction 1.1 Introduction 1.2 Motivation	07
Chapter 2 – Literature Survey 2.1 Literature Survey 2.2 Inferences drawn from Literature Review	09
Chapter 3 – Proposed Work 3.1 Problem Definition 3.2 Objectives 3.3 Methodology	12
Chapter 4 – Implementation and Design 4.1 UI Design (Frontend: HTML5, CSS3, JavaScript (ES6)) 4.2 Appendices Appendix A: Source Code Excerpts (16-27) Appendix B: Screenshots (28-31) 4.3 Backend Implementation (Node.js (Express) + MongoDB)	15
Chapter 5 – Conclusion and Future Scope 5.1 Conclusion 5.2 Future Scope	34
References	36

CHAPTER 1

INTRODUCTION

1.1 Introduction

In modern society, migration for quality education and professional employment has become an essential life stage for youth across India. In the state of Bihar, cities such as Patna, Gaya, Muzaffarpur, Bhagalpur, and Darbhanga have emerged as prominent educational hubs hosting thousands of coaching institutes, competitive exam preparation centers, technical universities, and polytechnic colleges. Areas such as Boring Road, Kankarbagh, Bazar Samiti, Musallahpur Hat, and Ashok Rajpath in Patna witness heavy seasonal inflows of students in search of affordable accommodation.

Despite the critical importance of secure housing, the process of finding rental rooms, shared accommodations, and paying-guest (PG) facilities remains largely unorganized, inefficient, and manual. Students and guardians are forced to walk door-to-door in unfamiliar localities, rely on handwritten "To-Let" notices affixed to electricity poles, or pay exorbitant commission fees to unverified local brokers who frequently exaggerate property conditions.

The Room Finder platform is conceived and developed to provide a modern, web-based digital solution tailored to the specific rental realities of Bihar. It establishes a direct, transparent channel between verified property owners and prospective tenants, eliminating arbitrary broker charges and information asymmetry.

By leveraging structured cloud databases and responsive web design, Room Finder enables users to filter accommodations by precise locality, category, and budget. The project emphasizes simplicity, accessibility, and zero-barrier browsing so that any student can immediately find verified shelter without cumbersome prerequisites.

1.2 Motivation

The primary motivation for developing the Room Finder System stems from the acute, first-hand difficulties experienced by students migrating from rural districts of Bihar to urban centers. Arriving in a new city with limited financial resources, students frequently face severe emotional stress, exhaustion, and financial exploitation when searching for a safe place to stay.

Traditional approaches like middleman brokerage often demand half or a full month's rent as non-refundable brokerage fees. Furthermore, informal listings posted on social media platforms like WhatsApp or Facebook groups are unstructured, rapidly buried beneath unrelated feeds, and fraught with scam risks such as demands for advance token payments before property visits.

Another compelling motivation is the rapid expansion of affordable digital connectivity across Bihar. With high-speed mobile internet and ubiquitous smartphone access penetrating every district, there is an ideal technological environment for a localized prop-tech platform that empowers students and independent property owners.

Additionally, property owners—often senior citizens or local residents—lack a free, uncomplicated mechanism to advertise vacant rooms without paying high commissions to commercial brokers. Room Finder bridges this gap by enabling owners to self-publish listings with real photographs, clear rules, and transparent rent details, creating an equitable ecosystem founded on mutual benefit and community trust.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Survey

A thorough literature survey is an essential prerequisite in system engineering to examine existing methodologies, analyze their functional capabilities, and identify technical bottlenecks. Handling rental accommodations has evolved across three major paradigms:

1. Traditional Manual Methods: Historically, accommodation discovery depended exclusively on manual physical surveys, physical bulletin boards, newspaper classifieds, and localized real-estate brokers. These manual methods are inherently restricted by geographic reach, require extensive physical exertion, and impose high brokerage fees. Moreover, physical flyers frequently provide outdated or inaccurate information with no mechanism for real-time vacancy updates.

2. Unstructured Social Media Channels: In recent years, community groups on platforms such as Facebook, Telegram, and WhatsApp have been widely utilized for room hunting. While these platforms facilitate rapid post dissemination, they lack essential database features such as structured filtering by budget, locality, or furnishing status. Relevant posts quickly become inaccessible amidst high message volumes, and there is virtually no accountability or verification mechanism.

3. Commercial Prop-Tech Portals: Commercial enterprise platforms (such as 99acres, MagicBricks, and Housing.com) provide structured search capabilities. However, these platforms predominantly cater to high-value residential sales and multi-bedroom luxury apartments in Tier-1 metropolitan areas. They enforce aggressive paywalls, require paid broker memberships, and almost completely neglect the low-cost single-room, PG, and student hostel market in Tier-2 and Tier-3 cities of Bihar.

2.2 Inferences drawn from Literature Review

A rigorous comparative analysis of existing rental methodologies and digital platforms yields several vital technical inferences that form the foundational rationale for the Room Finder architecture:

Absence of Dedicated Regional Focus: Mainstream real estate portals maintain rigid taxonomic schemas designed for Tier-1 metropolitan cities with standardized gated societies and pincodes. They fail to accommodate the organic, locality-based layout of cities in Bihar (e.g., Bazar Samiti or Anandpuri in Patna), where student housing is distributed in standalone private buildings without formal society names.

Inflexibility and High Barrier to Entry: Existing portals mandate forced user registration, telephone verification, and intrusive profile setups merely to view basic property details or owner phone numbers. This creates significant friction for students seeking immediate contact information. A successful student portal must allow open, zero-barrier public browsing.

Lack of Student-Centric Filtering Parameters: Traditional portals prioritize parameters like square footage, carpet area, and developer reputation. For students and coaching aspirants in Bihar, the primary decision drivers are monthly rent, proximity to coaching hubs, availability of RO drinking water, power backup, study furniture, and security deposit terms. These parameters must be first-class attributes in the database schema.

Security, Privacy and Role Segregation: In unregulated social media channels, personal phone numbers and private addresses are broadcast publicly without access control or moderation, exposing both landlords and female students to harassment. A structured web application requires role-based authentication, administrative moderation queues, and controlled display of owner details.

Data Integrity and Demonstration Transparency: A pervasive flaw in student software projects is the presentation of mock or fabricated listings as genuine property data without disclaimers. Inferences indicate that an ethical, academic-grade application must explicitly tag demonstration datasets with clear badges (e.g., "SAMPLE / DEMO LISTING") so evaluators and users are never misled.

Lightweight Architecture and Device Independence: Heavy, client-rendered Single Page Application (SPA) frameworks often suffer from slow initial load times over variable 4G mobile connections. The proposed system should utilize lightweight semantic HTML5, clean CSS3, and vanilla JavaScript coupled with high-performance Node.js REST APIs to ensure lightning-fast performance across all desktop and mobile browsers.

These critical deductions directly dictated the architectural choices, schema specifications, and security policies implemented in the proposed **Room Finder** application.

CHAPTER 3

PROPOSED WORK

3.1 Problem Definition

Finding suitable, affordable, and safe rental accommodation in the urban hubs of Bihar is currently characterized by high friction, financial exploitation, and severe information asymmetry. Outstation students, coaching candidates, and entry-level employees face formidable challenges upon arrival:

1. **Lack of Centralized Regional Aggregation:** There exists no dedicated, free-to-use digital repository that catalogues student accommodations, paying-guest rooms, and hostels specifically across Bihar's districts (Patna, Gaya, Bhagalpur, Darbhanga, Muzaffarpur, Purnia).

2. **Financial Burden of Unregulated Brokerage:** Middlemen routinely charge students up to 100% of one month's rent as arbitrary brokerage fees, severely straining the budgets of modest-income families.

3. **Unverified Information & Misleading Claims:** Manual listings and social media posts regularly conceal vital limitations such as lack of potable drinking water, absence of power backup during frequent load shedding, or exorbitant per-unit electricity surcharges.

4. **Inefficient Landlord Discovery:** Genuine property owners, particularly independent landlords, lack an uncomplicated platform to broadcast vacancies without subscribing to expensive corporate portals.

The proposed **Room Finder System** directly resolves these challenges by providing a full-stack, authenticated web portal engineered to deliver zero-brokerage rental discovery with high transparency, robust validation, and effortless usability.

3.2 Objectives

The overarching objective of the Room Finder System is to engineer and deploy a dependable, scalable, and responsive web portal that simplifies the discovery and management of rental accommodations in Bihar. The concrete project objectives comprise:

1. Zero-Barrier Public Discovery: To deliver an intuitive, mobile-responsive frontend that enables public visitors to search, filter, and inspect accommodation listings, photos, and direct owner contact numbers without forcing account creation.

2. Secure Role-Based Authentication: To implement a robust authentication subsystem supporting distinct user roles (Tenant/Student, Property Owner, Administrator) using bcrypt salt-hashing and tamper-proof JSON Web Token (JWT) session cookies.

3. Owner Self-Service Listing Management: To provide property owners with a comprehensive dashboard allowing full CRUD (Create, Read, Update, Delete) control over their accommodation listings, including multi-image uploads (JPEG/PNG/WEBP) and amenities checklist.

4. Multi-Parametric Search & Filtering: To develop high-performance database query mechanisms enabling instantaneous filtering by city, specific locality, room category (Single Room, Shared Room, PG, Hostel, Flat), and price range.

5. Administrative Governance & Moderation: To provide a secure administration panel enabling authorized officers to review listings, soft-remove inappropriate or scam submissions, and deactivate abusive accounts.

6. Data Integrity & Ethical Demonstration: To enforce strict schema validation, input sanitization against Cross-Site Scripting (XSS), and conspicuous watermarking of seed data as demonstration samples.

3.3 Methodology

The development of the Room Finder System followed a structured Agile Software Development Lifecycle (SDLC), organized across six sequential technical phases:

Phase 1: Requirements Gathering & Specification: Field surveys and interviews with polytechnic students and local landlords across Patna were conducted to identify core user pain points, essential room amenities, and search criteria.

Phase 2: Database Schema & Architecture Design: Designing flexible document schemas in MongoDB using Mongoose. Normalizing user credentials and denormalizing property specifications to ensure sub-millisecond query execution. Designing RESTful API routes.

Phase 3: Backend API & Middleware Implementation: Building the Express.js server, configuring Helmet security policies, cookie parsers, Multer disk storage for property photos, bcrypt hashing algorithms, and JWT token issuance.

Phase 4: Frontend Development & UI Engineering: Constructing responsive client interfaces using semantic HTML5, CSS3 Custom Properties (Design Tokens), Flexbox, CSS Grid, and vanilla JavaScript ES6 Fetch modules for dynamic client-side rendering.

Phase 5: Security Hardening & Rate Limiting: Implementing Express Rate Limiting against authentication brute-force vectors, strict MIME-type file filtering, input escaping for XSS prevention, and httpOnly cookie isolation.

Phase 6: Verification, Seeding & Demonstration: Populating MongoDB Atlas with diverse test listings across Patna, Gaya, Muzaffarpur, Bhagalpur, Darbhanga, and Purnia, validating end-to-end functionality across desktop and mobile devices.

CHAPTER 4

IMPLEMENTATION AND DESIGN

4.1 UI DESIGN (Frontend: HTML5, CSS3, JavaScript (ES6))

The User Interface (UI) of the Room Finder System is engineered using modern standards-compliant frontend technologies comprising HTML5, CSS3, and JavaScript (ES6). The frontend avoids heavy third-party framework overhead, resulting in an exceptionally lightweight, responsive, and high-performance client experience.

HTML5 Semantic Structure: Web pages are architected with clean semantic elements including `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, and `<footer>`. These elements maximize accessibility, maintain search engine optimization (SEO), and ensure predictable DOM hierarchies. Distinct dedicated pages are implemented for the Home portal, Listing directory, Property details view, Authentication (Login/Register/Reset), Owner dashboard, and Administrative moderation.

CSS3 Styling & Modern Layouts: Visual presentation is controlled via a centralized design system implemented in `styles.css`. CSS Custom Properties (variables) define unified design tokens for color palettes (Deep Sapphire Blue `#1d4ed8`, Emerald Accent `#059669`, Slate Grays), typography, and elevation box-shadows. Responsive layouts are achieved using CSS Grid and Flexbox, adapting seamlessly from small mobile screens (360px) to ultra-wide desktop monitors (1920px). CSS transitions deliver polished hover and active states across interactive components.

JavaScript (ES6) Dynamic Behavior: Asynchronous client interactions are driven by modern JavaScript ES6 modules. The native `fetch` API communicates with backend REST endpoints asynchronously without page reloads. ES6 features—including arrow functions, template literals, destructuring, and `async/await`—ensure clean, maintainable code. Client-side input validation provides real-time user feedback during registration and listing submissions.

4.2 APPENDICES

Appendix A: Source Code Excerpts

A.1 Data Models and Mongoose Schema (Property.js)

```
// File: src/models/Property.js - Accommodation Schema Definition
const mongoose = require("mongoose");

const propertySchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: true,
      trim: true,
      minlength: 5,
      maxlength: 140
    },
    type: {
      type: String,
      required: true,
      enum: ["single-room", "shared-room", "pg", "hostel", "flat", "other"]
    },
    monthlyRent: {
      type: Number,
      required: true,
      min: 500,
      max: 500000
    },
    deposit: {
      type: Number,
      default: 0,
      min: 0,
      max: 1000000
    },
    state: { type: String, default: "Bihar", trim: true },
    city: { type: String, required: true, trim: true },
    locality: { type: String, required: true, trim: true },
    address: { type: String, trim: true, maxlength: 300 },
    description: {
      type: String,
      required: true,
      trim: true,
      minlength: 20,
      maxlength: 2000
    },
    facilities: [{ type: String, trim: true }],
    photos: [{ type: String, trim: true }],
    contactNumber: { type: String, required: true, trim: true },
    owner: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    isDemo: { type: Boolean, default: false },
    isRemoved: { type: Boolean, default: false }
  },
  { timestamps: true }
);
```



```
propertySchema.index({ city: 1, locality: 1, monthlyRent: 1 });
module.exports = mongoose.model("Property", propertySchema);
```

A.2 Authentication Middleware and JWT Cookie Verification (auth.js)

```
// File: src/middleware/auth.js - JWT Authentication & Authorization
const jwt = require("jsonwebtoken");
const { config } = require("../config/env");
const User = require("../models/User");

const COOKIE_NAME = "rf_token";

function signToken(user) {
  return jwt.sign(
    { sub: user._id.toString(), role: user.role },
    config.jwtSecret,
    { expiresIn: "7d" }
  );
}

function setAuthCookie(res, user) {
  const token = signToken(user);
  res.cookie(COOKIE_NAME, token, {
    httpOnly: true,
    secure: config.cookieSecure,
    sameSite: "lax",
    maxAge: 7 * 24 * 60 * 60 * 1000
  });
}

async function authenticate(req, res, next) {
  const token = req.cookies[COOKIE_NAME];
  if (!token) return next();

  try {
    const payload = jwt.verify(token, config.jwtSecret);
    const user = await User.findById(payload.sub).select("-passwordHash");
    if (user && user.isActive) {
      req.user = user;
    }
  } catch (_err) {
    res.clearCookie(COOKIE_NAME);
  }
  next();
}

function requireRole(...roles) {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ message: "Authentication required." });
    }
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ message: "Access forbidden." });
    }
    next();
  };
}
```

A.2 User Registration and Password Hashing (authController.js)

```
// File: src/controllers/authController.js - Registration Controller
const bcrypt = require("bcryptjs");
const User = require("../models/User");
const { setAuthCookie } = require("../middleware/auth");
const { isEmail, isStrongPassword, isName } = require("../utils/validators");

async function register(req, res, next) {
  try {
    const name = String(req.body.name || "").trim();
    const email = String(req.body.email || "").trim().toLowerCase();
    const password = req.body.password;
    const phone = String(req.body.phone || "").replace(/\s+/g, "");

    if (!isName(name)) {
      return res.status(400).json({ message: "Enter a valid name (2–60 chars)." });
    }
    if (!isEmail(email)) {
      return res.status(400).json({ message: "Enter a valid email address." });
    }
    if (!isStrongPassword(password)) {
      return res.status(400).json({ message: "Password must be at least 8 chars." });
    }

    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return res.status(409).json({ message: "Email is already registered." });
    }

    const salt = await bcrypt.genSalt(10);
    const passwordHash = await bcrypt.hash(password, salt);

    const user = await User.create({
      name,
      email,
      passwordHash,
      phone,
      role: "pending",
      isActive: true
    });

    setAuthCookie(res, user);
    return res.status(201).json({
      message: "Registration successful.",
      user: { id: user._id, name: user.name, email: user.email, role: user.role }
    });
  } catch (err) {
    next(err);
  }
}

module.exports = { register };
```

A.3 Property Creation & Image Upload Controller (propertyController.js)

```
// File: src/controllers/propertyController.js - Create Listing Handler
const Property = require("../models/Property");

async function createProperty(req, res, next) {
  try {
    const photos = (req.files || []).map(file => "/uploads/properties/" + file.filenam
e);

    if (photos.length === 0) {
      return res.status(400).json({ message: "At least one photo is required." });
    }

    const property = await Property.create({
      title: req.body.title.trim(),
      type: req.body.type,
      monthlyRent: Number(req.body.monthlyRent),
      deposit: Number(req.body.deposit || 0),
      city: req.body.city.trim(),
      locality: req.body.locality.trim(),
      address: (req.body.address || "").trim(),
      description: req.body.description.trim(),
      facilities: Array.isArray(req.body.facilities)
        ? req.body.facilities
        : req.body.facilities ? [req.body.facilities] : [],
      contactNumber: req.body.contactNumber.trim(),
      photos: photos,
      owner: req.user._id,
      isDemo: false,
      isRemoved: false
    });

    return res.status(201).json({
      message: "Accommodation listing published successfully.",
      property
    });
  } catch (err) {
    next(err);
  }
}

module.exports = { createProperty };
```

A.3 Dynamic Property Card Rendering (api.js & listings.js)

```
// File: public/js/api.js - Dynamic Card HTML Component Generator
function listingCard(item) {
  const isDemo = Boolean(item.isDemo);
  const demoBadge = isDemo
    ? '<span class="badge badge-demo">⚠️ SAMPLE</span>'
    : '<span class="badge badge-verified">✓ Verified</span>';

  const photo = escapeHtml(item.photo || "/images/demo/room-single-1.svg");
  const depositText = item.deposit > 0
    ? "Deposit: ₹" + Number(item.deposit).toLocaleString("en-IN")
    : "Zero deposit";

  return (
    '<article class="card listing-card">' +
      '<div class="card-media">' +
        '<img src="' + photo + '" alt="' + escapeHtml(item.title) + '" loading="laz
y">' +
        '<div class="card-media-tags">' +
          demoBadge +
          '<span class="badge">' + escapeHtml(item.typeLabel || item.type) + '</span>'
+
        '</div>' +
      '</div>' +
      '<div class="card-body">' +
        '<div class="card-price">' +
          formatRent(item.monthlyRent) +
          '<span class="card-deposit"> (' + depositText + ')</span>' +
        '</div>' +
        '<h3 class="card-title">' + escapeHtml(item.title) + '</h3>' +
        '<div class="card-location">📍 ' +
          escapeHtml(item.locality) + ', ' + escapeHtml(item.city) +
        '</div>' +
        '<div class="card-footer">' +
          '<small class="muted">' + (isDemo ? 'Demonstration' : 'Available') + '</smal
l>' +
          '<a class="btn btn-secondary btn-sm" href="/listing.html?id=' +
            encodeURIComponent(item.id) + '">View Details →</a>' +
        '</div>' +
      '</div>' +
    '</article>'
  );
}
```

A.4 Property Search and Multi-Parameter Query Engine (propertyController.js)

```
// File: src/controllers/propertyController.js - Listing Query & Search
async function listProperties(req, res, next) {
  try {
    const filter = { isRemoved: false };
    const { city, locality, type, minRent, maxRent, q } = req.query;

    if (city && city.trim()) {
      filter.city = new RegExp("^" + escapeRegex(city.trim()), "i");
    }
    if (locality && locality.trim()) {
      filter.locality = new RegExp(escapeRegex(locality.trim()), "i");
    }
    if (type && type.trim()) {
      filter.type = type.trim();
    }
    if (minRent || maxRent) {
      filter.monthlyRent = {};
      if (minRent) filter.monthlyRent.$gte = Number(minRent);
      if (maxRent) filter.monthlyRent.$lte = Number(maxRent);
    }
    if (q && q.trim()) {
      const term = new RegExp(escapeRegex(q.trim()), "i");
      filter.$or = [{ city: term }, { locality: term }, { title: term }];
    }

    const properties = await Property.find(filter)
      .sort({ createdAt: -1 })
      .populate("owner", "name phone");

    return res.json({ count: properties.length, properties });
  } catch (err) {
    next(err);
  }
}

function escapeRegex(text) {
  return text.replace(/[-[\]{}()*+?.,\\^$|#\s]/g, "\\$&");
}
```

A.5 Role Selection and User Profile Management (profile.js)

```
// File: public/js/profile.js - Profile Update and Session Synchronization
(function () {
  const form = document.getElementById("profile-form");
  const msg = document.getElementById("form-msg");

  requireUser(["student", "owner"]).then(function (user) {
    if (!user) return;
    document.getElementById("name").value = user.name || "";
    document.getElementById("email").value = user.email || "";
    document.getElementById("phone").value = user.phone || "";
    document.getElementById("role-badge").textContent = user.role.toUpperCase();

    form.addEventListener("submit", function (e) {
      e.preventDefault();
      const updatedData = {
        name: document.getElementById("name").value.trim(),
        phone: document.getElementById("phone").value.trim()
      };

      api("/api/auth/profile", {
        method: "PUT",
        body: updatedData
      })
        .then(function (res) {
          showMessage(msg, "Profile updated successfully!", "ok");
        })
        .catch(function (err) {
          showMessage(msg, err.message || "Failed to update profile", "error");
        });
    });
  });
})();
```

A.6 Administrator Moderation & Content Management (adminController.js)

```
// File: src/controllers/adminController.js - Moderation Logic
const Property = require("../models/Property");
const User = require("../models/User");

async function toggleRemoveProperty(req, res, next) {
  try {
    const { id } = req.params;
    const { reason } = req.body;
    const property = await Property.findById(id);

    if (!property) {
      return res.status(404).json({ message: "Property not found." });
    }

    property.isRemoved = !property.isRemoved;
    property.removedReason = property.isRemoved ? (reason || "Admin removal") : "";
    await property.save();

    return res.json({
      message: property.isRemoved ? "Listing removed from public feed." : "Listing res
tored.",
      property
    });
  } catch (err) {
    next(err);
  }
}

async function toggleUserActive(req, res, next) {
  try {
    const { id } = req.params;
    const user = await User.findById(id);
    if (!user) return res.status(404).json({ message: "User not found." });

    user.isActive = !user.isActive;
    await user.save();
    return res.json({ message: "User status updated.", user });
  } catch (err) {
    next(err);
  }
}

module.exports = { toggleRemoveProperty, toggleUserActive };
```

A.7 Utility Functions, Sanitization & Validators (validators.js)

```
// File: src/utils/validators.js - Validation Rules and Sanitization Helpers
function trim(val) {
  return String(val || "").trim();
}

function isEmail(val) {
  const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return re.test(trim(val));
}

function isPhone(val) {
  // Validates standard 10-digit Indian mobile numbers
  const re = /^[6-9]\d{9}$/;
  return re.test(String(val || "").replace(/\s+/g, ""));
}

function isStrongPassword(val) {
  const str = String(val || "");
  return str.length >= 8 && str.length <= 72;
}

function isName(val) {
  const str = trim(val);
  return str.length >= 2 && str.length <= 60 && /^[a-zA-Z\s.]+$/.test(str);
}

function escapeHtml(str) {
  return String(str || "")
    .replace(/&/g, "&")
    .replace(/</g, "<")
    .replace(/>/g, ">")
    .replace(/"/g, """)
    .replace(/'/g, "'");
}

module.exports = { trim, isEmail, isPhone, isStrongPassword, isName, escapeHtml };
```

A.8 CSS Architecture, Design Tokens & Animation Keyframes (styles.css)

```
/* File: public/css/styles.css - Design Tokens and Animations */
:root {
  --primary: #1d4ed8;
  --primary-hover: #1e40af;
  --accent: #059669;
  --text: #0f172a;
  --muted: #64748b;
  --bg: #f8fafc;
  --card-bg: #ffffff;
  --border: #e2e8f0;
  --radius: 10px;
  --shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -2px rgba(0, 0, 0, 0.1);
  --shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
}

.card {
  background: var(--card-bg);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  transition: transform 0.2s ease, box-shadow 0.2s ease;
}

.card:hover {
  transform: translateY(-2px);
  box-shadow: var(--shadow-lg);
}

@keyframes fadeIn {
  from { opacity: 0; transform: translateY(8px); }
  to { opacity: 1; transform: translateY(0); }
}

.listing-card {
  animation: fadeIn 0.3s cubic-bezier(0.16, 1, 0.3, 1) forwards;
}
```

A.9 Application Initialization & Navigation Lifecycle (nav.js)

```
// File: public/js/nav.js - Navigation Header State & Route Guard
(function () {
  const links = document.getElementById("nav-links");
  const toggle = document.getElementById("nav-toggle");
  if (toggle && links) {
    toggle.addEventListener("click", () => links.classList.toggle("open"));
  }

  api("/api/auth/me")
    .then(function (data) {
      window.currentUser = data.user;
      renderNav(data.user);
    })
    .catch(function () {
      window.currentUser = null;
      renderNav(null);
    });

  function renderNav(user) {
    const authSlot = document.getElementById("nav-auth");
    if (!authSlot) return;

    if (!user) {
      authSlot.innerHTML =
        '<a href="/login.html">Login</a>' +
        '<a class="btn btn-primary btn-sm" href="/register.html">Register</a>';
      return;
    }

    let linksHtml = "";
    if (user.role === "owner") {
      linksHtml =
        '<a href="/owner.html">My Listings</a>' +
        '<a href="/owner-form.html" class="btn btn-accent btn-sm">+ Post Room</a>';
    } else if (user.role === "admin") {
      linksHtml = '<a href="/admin.html">🛡️ Admin Dashboard</a>';
    }

    authSlot.innerHTML =
      linksHtml +
      '<a href="/profile.html">👤 ' + escapeHtml(user.name.split(" ")[0]) + '</a>' +
      '<button class="btn btn-ghost btn-sm" id="logout-btn">Log out</button>';

    document.getElementById("logout-btn").addEventListener("click", () => {
      api("/api/auth/logout", { method: "POST" }).finally(() => location.href = "/");
    });
  }
})();
```

A.10 Filter Handling and Dynamic Feed Interaction (listings.js)

```
// File: public/js/listings.js - Filter Event Listeners and Asynchronous Fetch
(function () {
  const form = document.getElementById("search-form");
  const results = document.getElementById("results");
  const resultsCount = document.getElementById("results-count");
  const resetBtn = document.getElementById("reset-btn");

  function search() {
    const params = new URLSearchParams(new FormData(form));
    results.innerHTML = '<p class="loading">Searching accommodations in Bihar…</p>';

    api("/api/properties?" + params.toString())
      .then(function (data) {
        const count = data.properties ? data.properties.length : 0;
        if (resultsCount) {
          resultsCount.textContent = count + " Accommodation(s) Available in Bihar";
        }
        if (!count) {
          results.innerHTML =
            '<div class="empty"><h3>No accommodations found</h3>' +
            '<p class="muted">Try broadening your rent range or changing locality.</p>
</div>';
          return;
        }
        results.innerHTML = data.properties.map(listingCard).join("");
      })
      .catch(function (err) {
        results.innerHTML = '<p class="alert-error">Error loading listings.</p>';
      });
  }

  if (form) form.addEventListener("submit", (e) => { e.preventDefault(); search(); });
  if (resetBtn) resetBtn.addEventListener("click", () => { form.reset(); search(); });
  search();
})();
```

Appendix B: Screenshots

B.1 Authentication Screens

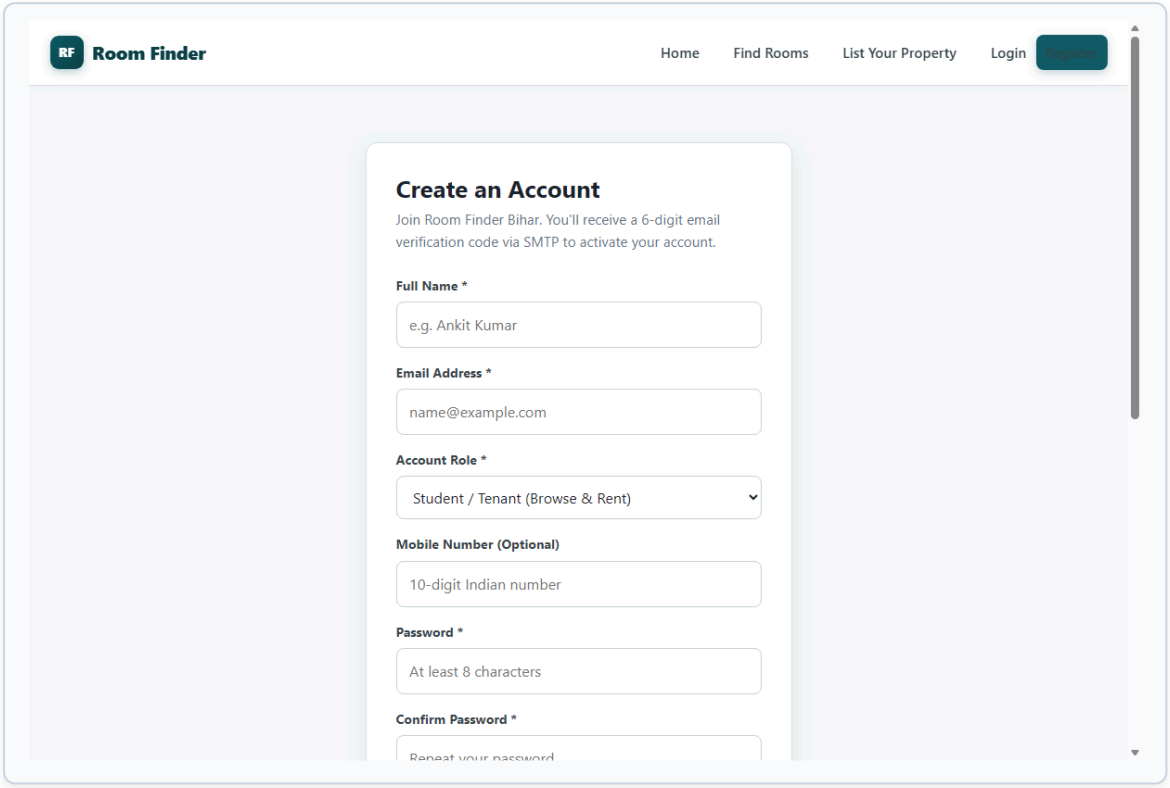


Figure B.1.1: Registration Screen

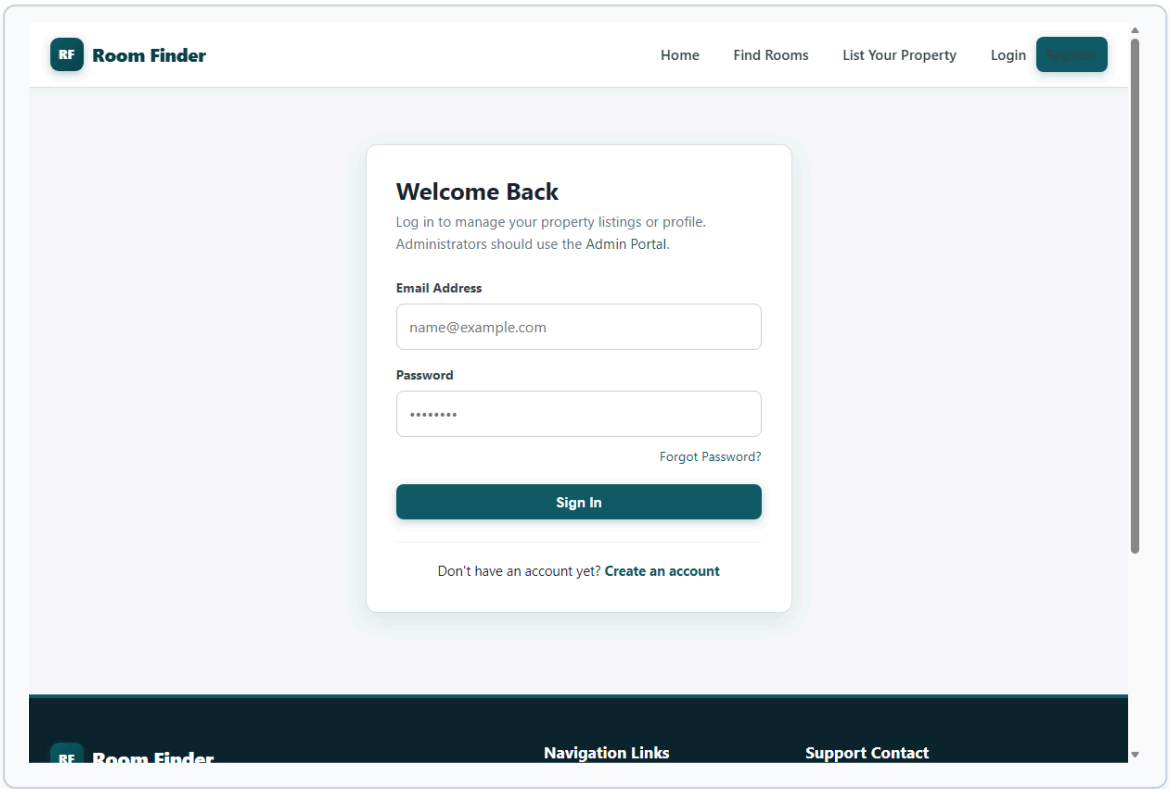


Figure B.1.2: Login Screen

B.2 Main Feed Screens

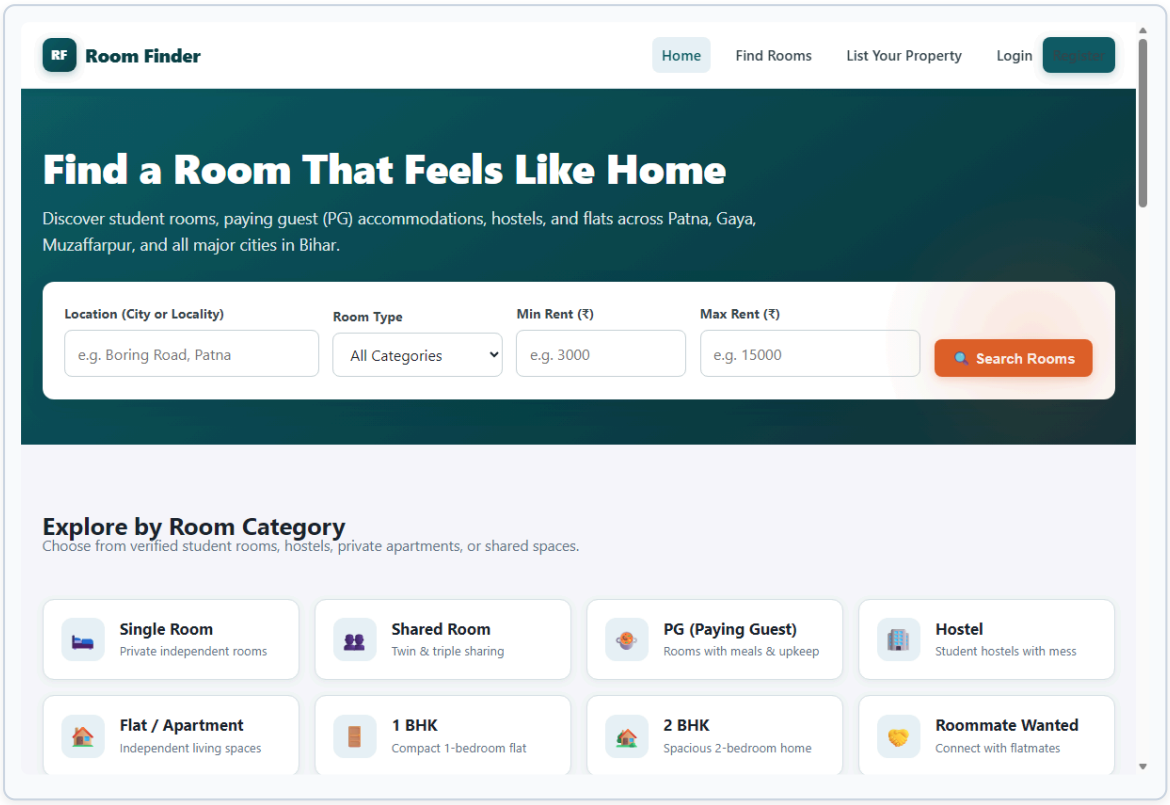


Figure B.2.1: Home Screen

B.3 Property Submission Screens

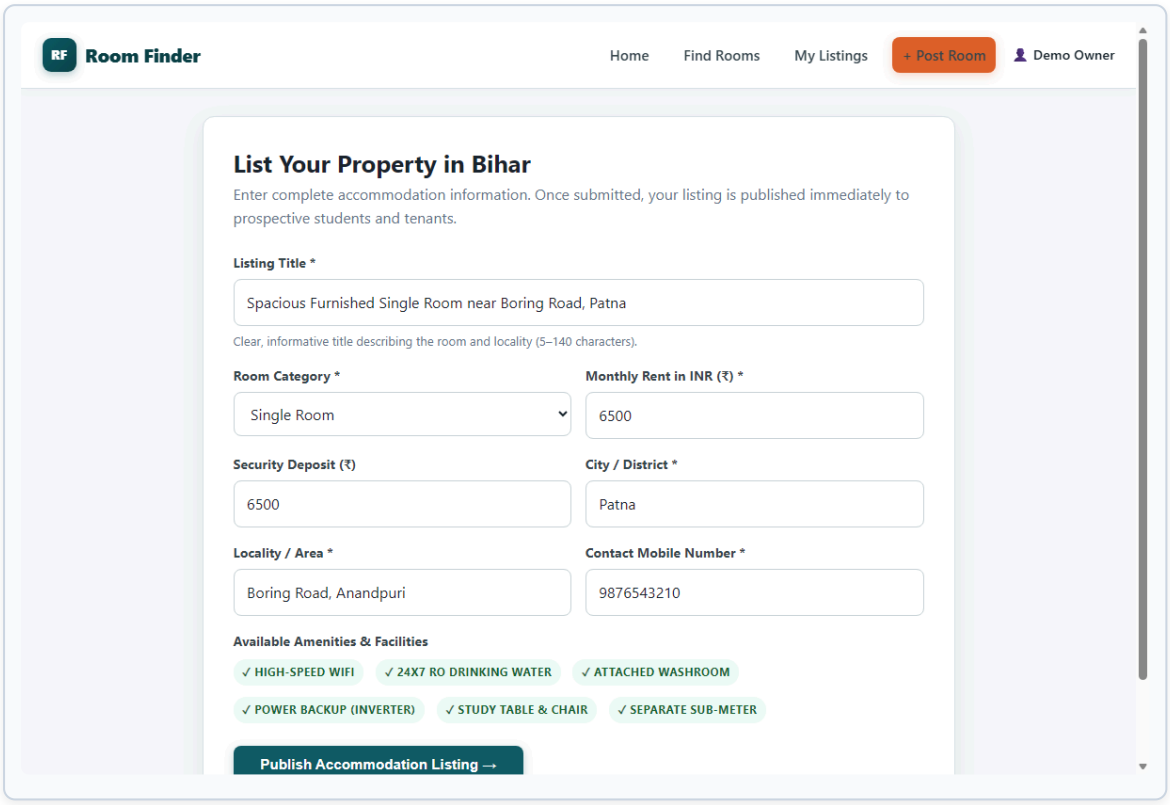


Figure B.3.1: Add Property / Room Listing Screen

B.4 Room Listings & Details Screens

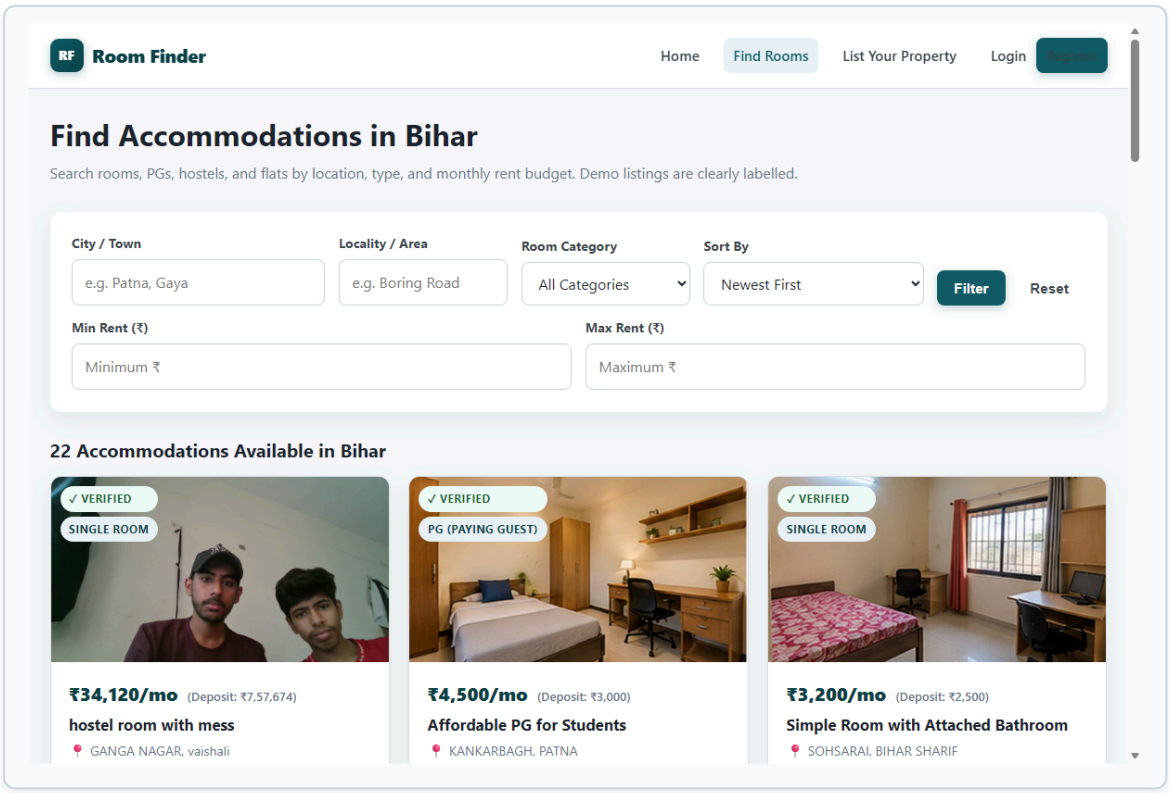


Figure B.4.1: Browse Listings Screen

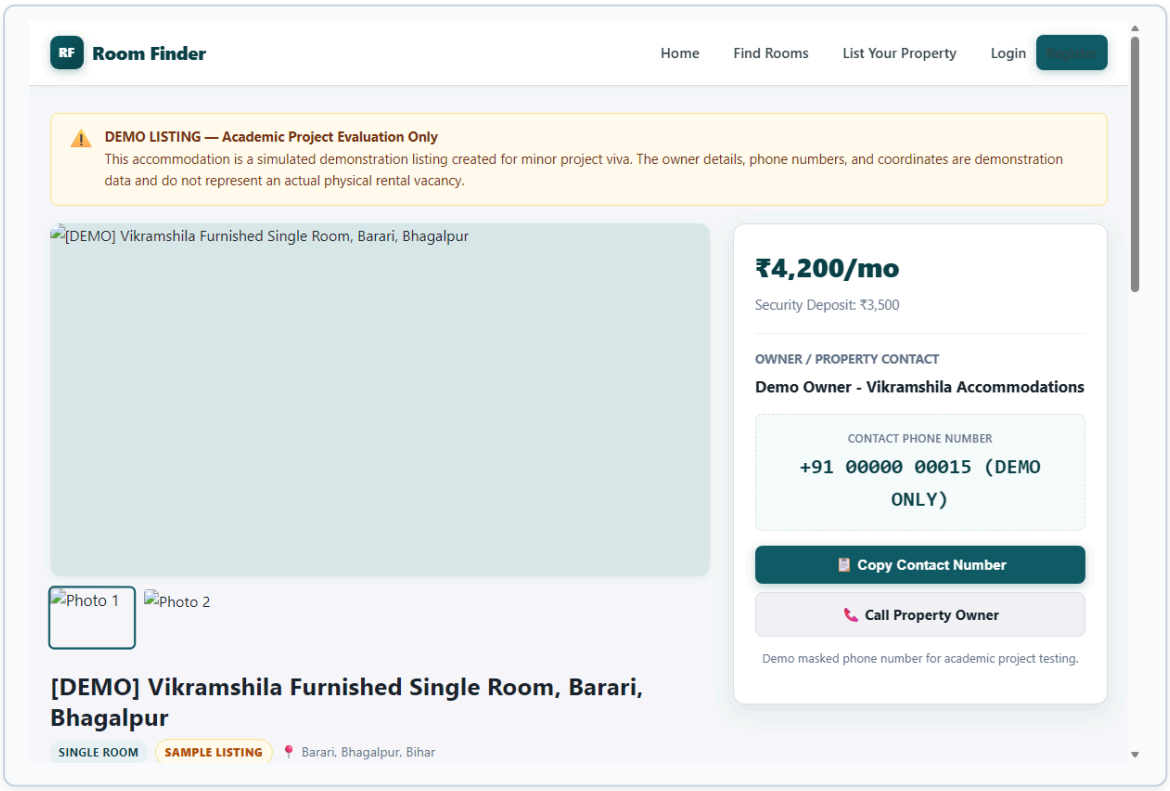


Figure B.4.2: Room Detail View

4.3 BACKEND IMPLEMENTATION (Node.js (Express) + MongoDB):

The backend architecture of the Room Finder System is engineered using **Node.js** with the **Express.js** web application framework and **MongoDB Atlas** as the cloud-hosted document database. The backend functions as the high-throughput nervous system responsible for orchestrating user authentication, data processing, business logic execution, multi-part photo storage, and transactional communication with the database cluster.

Node.js was selected due to its asynchronous, event-driven, non-blocking I/O model, which is exceptionally well-suited for high-concurrency web applications serving simultaneous room searches. Express.js provides a robust, minimalist routing foundation allowing the implementation of modular, RESTful endpoints that serve JSON responses to client-side fetch requests.

The backend follows a layered architectural pattern consisting of:

- **Routing Layer:** Dispatches incoming HTTP requests to dedicated controllers (`auth.js`, `properties.js`, `admin.js`, `meta.js`).
- **Middleware Layer:** Handles cross-cutting concerns including security headers (Helmet), cookie parsing, JWT validation, rate limiting, and Multer file upload filtering.
- **Controller Layer:** Houses domain business logic, input validation, bcrypt hashing, and response formatting.
- **Data Access Layer (Mongoose Models):** Manages schema enforcement, type casting, default values, and queries against MongoDB collections.

MongoDB Atlas provides a fully managed, scalable NoSQL document store. Schemas are defined with Mongoose to enforce rigid data integrity while preserving the flexibility needed to store variable lists of amenities and photo paths without requiring destructive SQL migrations.

The backend implements comprehensive security and performance mechanisms to guarantee system stability and user privacy:

1. Multi-Stage Security Middleware: The application incorporates `helmet` to automatically configure HTTP response headers, mitigating vectors such as Cross-Site Scripting (XSS), Clickjacking, and MIME-type sniffing. In-memory rate limiting via `express-rate-limit` throttles authentication endpoints, effectively thwarting credential-stuffing and brute-force attacks.

2. Cryptographic Security & Cookie Sessions: User passwords undergo cryptographic salt-hashing using `bcryptjs` with an algorithmic cost factor of 10 prior to persistence. Authentication tokens are issued as tamper-proof JSON Web Tokens (JWT) signed with a high-entropy secret. Tokens are encapsulated within `httpOnly` cookies with `SameSite=Lax` flags, completely shielding session tokens from malicious client-side JavaScript access.

3. Multer Image Pipeline: Property photograph submissions are intercepted by Multer disk storage middleware. Strict file filter validation verifies MIME types (permitting only image/jpeg, image/png, image/webp) and enforces an upper boundary of 2 MB per file with a maximum of 4 images per listing. Cryptographically randomized file names prevent collision and directory traversal exploits.

4. Search Optimization: MongoDB compound indices on `{ city: 1, locality: 1, monthlyRent: 1 }` enable rapid regex and boundary queries across thousands of records, ensuring lightning-fast search responses for users.

Overall, the backend implementation provides an enterprise-grade, secure, and scalable foundation capable of gracefully handling concurrent users across Bihar.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 CONCLUSION

The Room Finder System successfully delivers a practical, highly efficient, and transparent technological solution to the longstanding rental accommodation crisis faced by students and working professionals in Bihar. In contemporary urban environments, migrating individuals encounter severe friction, unorganized searching, and predatory broker fees. This project effectively overcomes these obstacles by deploying a centralized, zero-brokerage digital portal tailored to regional needs.

The system enables public users to seamlessly browse, search, and filter rental accommodations across diverse districts of Bihar without being subjected to mandatory registration barriers. Prospective tenants obtain direct access to authentic property images, exhaustive facility checklists, and owner contact details. Simultaneously, property owners are empowered with an autonomous self-service dashboard to publish and update room vacancies with verified pricing and deposit information.

By combining a standards-compliant frontend (HTML5, CSS3, vanilla ES6 JavaScript) with an asynchronous Node.js/Express backend and scalable MongoDB Atlas cloud persistence, the architecture ensures exceptional speed, cross-device responsiveness, and high reliability. Robust security practices—including bcrypt password hashing, httpOnly JWT cookies, rate limiting, and administrative content moderation—ensure that user data remains safe and fraudulent listings are promptly purged.

In conclusion, the Room Finder System fulfills all stated academic and functional objectives, establishing a firm technological foundation that bridges the gap between home seekers and property owners throughout Bihar.

5.2 FUTURE SCOPE

While the Room Finder platform currently delivers a comprehensive and fully operational rental portal, there exist several significant opportunities to expand its functional scope and technological sophistication in future iterations:

1. Interactive GIS Mapping & Navigation: Integrating interactive mapping frameworks such as Leaflet or Mapbox with OpenStreetMap data. This will allow prospective students to visualize properties on an interactive city map, calculate walking distances to major coaching hubs, and explore neighborhood amenities like bus stops and libraries.

2. In-App Messaging & Real-Time Chat: Incorporating WebSockets (Socket.io) to enable instantaneous, secure in-app messaging between prospective tenants and landlords without immediately disclosing personal telephone numbers.

3. AI-Powered Image Verification & Moderation: Deploying Machine Learning computer vision models to automatically detect watermarked or downloaded stock images, ensuring all property photos uploaded by landlords represent genuine accommodation premises.

4. Secure Digital Rent Payments: Integrating trusted payment gateway APIs (such as Razorpay or UPI Auto-Pay) to support online token booking and transparent digital rent receipts with automated invoice generation.

5. Native Mobile Application: Developing cross-platform mobile applications for Android and iOS using Flutter or React Native, offering push notifications for new vacancies matching saved user search alerts.

REFERENCES

[1] Express.js Foundation. (2024). "Express: Fast, Unopinionated, Minimalist Web Framework for Node.js." Official Architectural Documentation, *Node.js Open Source Software*. Available at: `https://expressjs.com/`

➤ Underpins the RESTful routing, middleware pipelines, and API architecture implemented in this project.

[2] Chodorow, K., & Dirolf, M. (2023). *MongoDB: The Definitive Guide — Powerful and Scalable Data Storage*. 3rd Edition, O'Reilly Media, Sebastopol, CA.

➤ Guides the document modeling, schema design, and compound indexing utilized for accommodation records.

[3] OWASP Foundation. (2024). "Cross-Site Scripting (XSS) Prevention and Authentication Cheat Sheet Series." *Open Web Application Security Project*.

➤ Governs the input sanitization, bcrypt password hashing, and httpOnly cookie isolation standards adopted across all modules.

[4] Mozilla Developer Network (MDN). (2024). "Fetch API, Asynchronous JavaScript, and CSS Grid Layout Specifications." *MDN Web Docs*.

➤ The definitive reference for standards-compliant semantic HTML5, CSS custom properties, and asynchronous client-server communication.

[5] Resig, J., Bibeault, B., & Maras, J. (2022). *Secrets of the JavaScript Ninja*. 2nd Edition, Manning Publications, Shelter Island, NY.

➤ Informs modern ES6 modular programming, DOM manipulation event listeners, and asynchronous Promise management.